

HelixConstitution

HelixConstitution

Універсальная інжынерная канстытуцыя, якую атрымлівае ў спадчыну кожны праект — закон супраць блефу, які прымушаецца механічна і распаўсюджваецца як адзін Git-субмодуль.

Зводка

HelixConstitution — гэта адзіны, незалежны ад праекта збор правілаў, які дадаецца як Git-субмодуль у кожны праект Helix/vasic-digital і кадзіруе непарушную інжынерную дысцыпліну (антыблеф, валідацыя толькі на падставе доказаў, бяспека даных і хоста, дакументацыя і пакрыццё тэстамі), распаўсюджваючы яе на парк з больш чым 140 рэпазіторыяў. Гэта кіраўнічы каркас, які забяспечвае ўзгодненасць усёй сістэмы.

Кароткае апісанне

Універсальны, спадчынны Constitution, які пастаўляецца як Git-субмодуль. Ён вызначае абавязковыя, непарушныя правілы — бар'еры супраць блефу на аснове доказаў, імунітэт да ілжыва-пазітыўных вынікаў, бяспеку даных і хоста, дысцыпліну пакрыцця тэстамі і дакументацыі — якія кожны праект, што выкарыстоўвае яго, атрымлівае аўтаматычна і можа пашыраць, але ніколі не аслабляць.

Падрабязнае апісанне

HelixConstitution — гэта кананічная, адзіная крыніца праўды для інжынерных практык, агульных для кожнага праекта, які далучаецца да яго праз даданне Git-субмодуля — закон інжынерыі, які распаўсюджваецца і версіюецца гэтак жа, як і код. Яго цэнтральны элемент — Constitution.md — гэта дакумент аб'ёмам каля 1 МБ, які бесперапынна версіюецца і

змяшчае нумараваныя артыкулы (сямейства пагадненняў §11.4.x, цяпер да §11.4.170), а таксама інструкцыі для асобных агентаў (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md), якія спасылаюцца на яго, каб і людзі, і кожны агент CLI карысталіся адным і тым жа зборнікам правілаў. Спадчыннасць арганізавана наўмысна ў тры ўзроўні: універсальная база (гэты субмодуль), праектны ўзровень (уласныя Constitution/CLAUDE/AGENTS праекта, якія яго пашыраюць) і дадатковы ўзровень для асобных каталогаў — якія ацэньваюцца зверху ўніз, дзе праект можа *пасупна* пацвердзіць правілы, але архітэктурна пазбаўлены магчымасці іх *аслабіць*. Вынік — парк з больш чым 140 рэпазіторыяў, якія не могуць ціха разысціся, бо дысцыпліна, якой яны падзяляюцца, зафіксавана, а не трымаецца ў памяці.

Дакумент безумоўна незалежны ад прадметнай вобласці: усе згадкі канкрэтных пастаўшчыкоў, мадэляў абсталявання, партоў ці версій бібліятэк павінны перамяшчацца ў уласны Constitution спажываючага праекта, а ўніверсальнасць ніколі не здаецца на веру — яна павінна быць *заслужанай* праз праходжанне спецыяльнага чатырохчасткавага тэсту, перш чым правіла дазваляецца ўключыць у базавы збор. Яго філасофскі каркас — антыблеф, выражаны праз сістэму ўзаемазвязаных пагадненняў: §1.1 імунітэт да ілжыва-пазітыўных вынікаў, §11.4 пагадненне аб якасці для канчатковага карыстальніка, §11.4.6 забарона на здагадкі, §11.4.69 таксанамія пазітыўных доказаў. Іх сумеснае дзеянне стварае адну жорсткую лінію: планка для выпуску прадукту ніколі не “тэсты прайшлі”, а “рэальны карыстальнік можа карыстацца функцыяй”, і кожны станоўчы вынік павінен спасылацца на зафіксаваныя фізічныя доказы, інакш ён не лічыцца. Дадатковы submodules-catalogue.md (142 рэпазіторыі) ператварае пытанне “ці ёсць у нас штосьці, што ўжо гэта робіць?” у рэфлекс “спярша праверым каталог, пашыраем — не перапісваем” яшчэ да напісання радка новага кода. Дапаможныя скрыпты знаходзяць субмодуль на любой глыбіні ўкладання і распаўсюджваюць кожны каміт на чатыры незалежныя Git-правайдары, так што адзіны аўтарытэтны збор правілаў становіцца немагчымым страціць.

Змесціва

Чаму мы яго стварылі

Шматлікія вялікія прадуктовыя дадаткі і дзясяткі раз’яднаных паўторна выкарыстоўваемых падмадуляў, аўтарства аднаго ўладальніка, няспынна паўтаралі адзін і той жа цяжка дасягнуты досвед — і няспынна сутыкаліся з адным і тым жа класам памылак: тэстамі і справаздачамі аб статусе, якія сцвярджаюць поспех, у той час як функцыя не працуе для канчатковага карыстальніка («ілюзіі ПРАЙШЛА» і «ілюзіі

НЕПРАЙШЛА»). Кожная крыміналістычная закладка ў Constitution фіксуе рэальны інцыдэнт (напрыклад, ілюзію ПРАЙШЛА аўдыё-маршрутызацыі D3 20 мая 2026 года, калі валідацыя стала зялёнай з пустым полем «Выкарыстоўваецца кодэк», ці гіганцкі UI-кнопкі ад 25 чэрвеня 2026 года, якія прайшлі тэсты на роўнасць токенаў, у той час як рэальны экран быў пашкоджаны). Constitution існуе, каб раз і назаўжды зрабіць гэты клас несумленнага поспеху механічна немагчымым — так, каб дысцыпліна не змянялася паміж праектамі і не забывалася паціху.

Чаму гэта пераварот

Ён ператварае інжынерную культуру з дакументацыі, якой спадзяюцца прытрымлівацца, у спадчыннае, версіяванае, механічна прымусовае права — розніцу паміж стылявым дапаможнікам і кампілятарам. Адзінае абнаўленне падмадуля адначасова і адсочвальна абнаўляе правілы для ўсёй сістэмы. Адзінае антыілюзійнае пагадненне *гарантавана* прысутнічае ў кожным рэпазіторыі-спажыву не праз давер, а праз канструкцыю: прапагандычны бар’ер літаральна правярае наяўнасць нумара пункта па ўсёй сістэме, а парны мутацыйны тэст даказвае, што сам бар’ер не ілюзія — такім чынам, нават прымус прымушаецца. Кіраванне перастае быць марываннем на вікі, якое ніхто не чытае, і становіцца аўдытаванай, тэсціраванай рэальнасцю, на якую можна накіраваць задачу CI.

Што новага

- **Constitution як падмадуль** — інжынернае права распаўсюджваецца і версіянуецца дакладна як код, з наўмыснымі тэгамі тыпу v1.0.0 і прывязкай да праекта, каб кожны рэпазіторый ведаў *дакладна*, якой версіі закона ён падпарадкоўваецца.
- **Антыілюзія як першачарговая крыміналістычная дактрына** — кожны пункт вядзе да літаральнага распараджэння апэратара і, часта, да канкрэтнага рэальнага інцыдэнту, які яго выклікаў, таму збор правілаў чытаецца як прэцэдэнтнае права, а не як меркаванне.
- **Мета-тэсціраванне ўласных правілаў (§1.1)** — кожны бар’ер суправаджаецца мутацыяй, якая павінна пераключыць ПРАЙШЛА→НЕПРАЙШЛА, таму «бар’ер не імітацыя» не сцвярджаецца, а даказваецца пры кожным запуску; бар’ер, які ніколі не можа праваліцца, лічыцца горш за адсутнасць бар’ера наогул.
- **Заробленая ўніверсальнасць** — спецыяльны чатырохчасткавы тэст вызначае, ці з’яўляецца правіла сапраўды ўніверсальным, ці толькі спецыфічным для

праекта, захоўваючы базавы набор лаканічным, пераносным і свабодным ад уплыву вендараў.

Як гэта выкарыстоўваецца ва ўсіх прадуктах (магчымасці, якія яно дае)

Як абавязковая кіраўнічая апора, HelixConstitution — гэта не дакумент, з якім кансультавацца ў сямействе, а апорная канструкцыя, на якой яно будзеца:

- **Апорны каркас кіравання:** кожны праект Helix/vasic-digital дадае яго як падмадуль і імпартуе з CLAUDE.md / AGENTS.md / QWEN.md ці ўласнага Constitution.md; правілы дзейнічаюць безумоўна, з самага першага каміту, без магчымасці адмовіцца для асобнага праекта.
- **Бар’еры і патрабаванні:** ён вызначае чатырохузроўневую мадэль пакрыцця — крыніца прысутнічае, вытрымлівае зборку, паводзіць сябе ў рантайме, бар’ер не ілюзія — якую павінна прайсці функцыя на ўсіх чатырох узроўнях, перш чым лічыцца завершанай, а таксама пашыраны спіс пазначаных патрабаванняў: апрацоўка дадзеных аўтэнтыфікацыі (§11.4.10), заўсёды сінхранізаваная дакументацыя (§11.4.60), мандат на кантэйнеры-падмадулі (§11.4.76), CodeGraph (§11.4.78), абавязковае пакрыццё тыпаў тэстаў (§11.4.169) і іншыя.
- **Распаўсюджванне:** бар’еры CM-COVENANT-114-NNN-PROPAGATION сцвярджаюць літаральную наяўнасць тэксту пункта ва ўсёй сістэме-спажыўцу, каб пагадненне не магло быць ціха знята ў адным куце маёнтка; неадпаведнасць з’яўляецца жорсткім блакатарам рэлізу без магчымасці абмінуць яго праз сцягі.
- **Адкрыццё:** submodules-catalogue.md ператварае пытанне «ці ёсць у нас ужо нешта, што рабіць X?» у адказ з аднаго погляду, перш чым будзе створаны новы модуль, знішчаючы дубляванне намаганняў у корані.
- **Аднастайнасць агентаў AI:** адзін і той жа закон аднолькава выражаецца для кожнага агента CLI (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi праз AGENTS.md, Qwen Code праз QWEN.md), таму незалежна ад таго, які інструмент дакранаецца да кода, ён падпарадкоўваецца аднаму і таму ж пагадненню.

Змест

Найбуйнейшыя тэхнічныя праблемы і спосабы іх вырашэння

- **Пошук субмодуля з адвольнай глыбіні ўкладання** — правіла, схаванае на трох узроўнях субмодуляў, усё роўна павінна знайсці закон, не ведаючы, дзе ён знаходзіцца → `find_constitution.sh` рухаецца ўверх па бацькоўскіх каталогах і рэкурсіўна адсочвае ўказальнік `git-суперпраекта`, улічваючы пераазначэнне `CONSTITUTION_DIR` і два падтрыманыя фарматы размяшчэння (`constitution/`, `submodules/constitution/`), таму вырашэнне застаецца дэтэрмініраваным незалежна ад глыбіні ўкладання.
- **Падтрыманне аднаго аўтарытэтнага рэпазіторыя на чатырох Git-правайдэрах** — люстэркі бескарысныя, калі яны разыходзяцца → `install_upstreams.sh` чытае дэкларатыўныя аддаленыя рэпазіторыі `Upstreams/*.sh` і настройвае `origin` з некалькімі URL-адрасамі для `push`, таму адзіная каманда `git push` атамна распаўсюджваецца на GitHub (асноўны), GitLab, GitFlic і GitVerse, і ні адно люстэрка не адстае.
- **Прадухіленне разрастання правілаў і прасочвання праектаў у ўніверсальную базу** — кожнае спакуслівае “проста дадай гэта сюды” падрывае пераноснасць → да кожнага новага правіла прымяняецца чатырохчасткавы тэст заробленай універсальнасці разам з класіфікацыяй універсальнае-праектнае паводле §11.4.17, што вымушае праектна-спецыфічныя пытанні вяртацца на ўзровень праекта, дзе ім і належыць быць.
- **Праверка, што механізм наследавання сапраўды працуе** — бар’ер, які ніколі не правальваецца, — гэта бар’ер, якому нельга давяраць → `meta_test_inheritance.sh`, вартавы мета-тэст, наўмысна выдаляе якару §11.4 і сцвярджае, што бар’ер яе лавіць, таму механізм прымусу бесперапынна паўторна правяраецца на магчымыя нябачныя паломкі.

Тэхналагічны стэк

- **Спадчыннасць праз Git-субмодулі** — чаму: Git-субмодулі — адзіны механізм, які дазваляе зборніку правіл быць аўтарытэтным і версійна зафіксаваным для кожнага спажыўца, абнаўляцца праз відавочны, разглядаемы каміт замест ціхага капіравання; як: праекты-спажыўцы дадаюць субмодуль і `@import` яго агенцкія файлы, а тры ўзроўні ацэньваюцца зверху ўніз з дакладнай дамовай “пашырае, а не аслабляе” на кожнай мяжы.
- **`find_constitution.sh`** — чаму: правілы бескарысныя, калі глыбока ўкладзены код не можа надзейна іх знайсці, а жорстка прапісваючы шляхі, можна зламаць усё, калі

праект перабудуюць; *як*: рух па бацькоўскіх каталогах разам з рэкурсіяй `git rev-parse --show-superproject-working-tree`, падтрыманы пераазначэннем `CONSTITUTION_DIR`, што вырашае абодва падтрыманыя фарматы размяшчэння.

- **install_upstreams.sh + Upstreams/** — *чаму*: чатырохправайдэрская рэзервовасць рэальная толькі тады, калі не патрабуе дадатковых высілкаў для падтрымання, інакш люстэркі гніюць; *як*: дэкларатыўныя файлы `.sh` для кожнага аддаленага рэпазіторыя пераўтвараюцца ў адзіны `origin` з некалькімі URL, што зводзіць чатыры `push`-аперацыі да адной.
- **Мета-тэсты на мутацыю §1.1** — *чаму*: бар’ер, які ніколі не правальваецца, горш за яго адсутнасць, бо стварае ілюзію надзейнасці; *як*: кожны бар’ер спараны з мутацыяй праз `sed`/перайменаванне, якая павінна перавесці `PASS`→`FAIL`, пасля чаго аднаўляецца, таму кожны бар’ер пацвярджае, што ён усё яшчэ працуе пры кожным запуску.
- **Бар’еры распаўсюджвання (CM-COVENANT-114-NNN-PROPAGATION)** — *чаму*: пагадненне універсальнае толькі тады, калі яно праверачна прысутнічае ва ўсіх спажывацях, а не толькі ў флагманскім рэпазіторыі; *як*: літаральны пошук нумара пункта па спажывацях, падмацаваны спараным мета-тэстам §1.1, які пацвярджае, што сама праверка распаўсюджвання можа праваліцца.
- **submodules-catalogue.md (§11.4.74)** — *чаму*: найхутчэйшы спосаб парушыць дысцыпліну антыдубліравання — не ведаць, чым ты ўжо валодаеш; *як*: пералік 142 рэпазіторыяў, згрупаваных па магчымасцях, з праверкай каталога ў трэкеры *да* таго, як будзе створаны новы каркас.
- **Экспарт у некалькіх фарматах** — *чаму*: адзін і той жа закон павінен аднолькава спажывацца людзьмі, якія яго чытаюць, інструментамі, якія яго разбіраюць, і архівамі, якія яго захоўваюць; *як*: кожны кананічны дакумент экспартуецца ў фарматах `.md` / `.html` / `.pdf` / `.docx` з адной крыніцы.

Змест

Заўвагі аб статусе і сумленнасці

- **Статус: адпраўлена.** Актыўна версіюецца і выкарыстоўваецца як падмадуль па ўсёй інфраструктуры (публічныя кананічныя і люстэркавыя рэпазіторыі).
- **Ліцэнзія: будзе вызначана** — у прагледжаных зыходных матэрыялах не пазначана; пераканайцеся ў адпаведнасці з файлам `LICENSE` рэпазіторыя перад публікацыяй.
- **Дадатковыя люстэркавыя рэпазіторыі ў апстрыме:** `GitLab helixdevelopment1/helixconstitution`, `GitFlic helixdevelopment/helixconstitution`, `GitVerse helixdevelopment/HelixConstitution`.

Прыярытэтны ўзровень: Helix-пярвічны — абавязковая кіраўнічая аснова, на якой будзецца ўсё ў сямействе Helix.